

Contents

1	Overall Project Architecture	4
2	Input Data and Temporal Alignment	5
2.1	Vehicle and Task Records	5
2.2	Streaming Input and Timestamp Alignment	6
3	VEC Environment and Resource State	7
3.1	Environment Configuration	7
3.2	Edges, UAVs, and Processing Queues	8
3.3	Queue Servicing	9
4	UAV Control and Repositioning	10
4.1	Traffic Hotspot Extraction with K-Means	10
4.2	Center Assignment and Bounded Movement	10
4.3	Connecting UAV Control to the Environment Timeline	11
5	Action Space and Hard Constraints	12
5.1	Action-Space Construction	12
5.2	Action Mask as a Hard Constraint	13
6	Communication Model and Action-Outcome Computation	14
6.1	Link Rate	14
6.2	Processing Time and Energy	15
6.3	Composition of Route Delays	16
7	Construction of Observation and Global State	17
7.1	Global State for the Centralized Critic	17
7.2	Agent Observation	18
8	Reward and Single-Decision Dynamics	19

9	MASAC Agent Architecture	20
9.1	Precise Definition of the Algorithm in This Project	20
9.2	Hyperparameters and MLP Structure	21
9.3	Replay Buffer	22
9.4	Actor, Critics, and Target Critics	23
9.5	Policy Masking and Action Selection	24
9.6	Construction of Probability and Log-Probability	25
10	Mathematical Training of MASAC	26
10.1	1. Critic Prediction for the Actual Action	26
10.2	2. Soft Value of the Next State	26
10.3	3. Bellman Target and Critic Training	27
10.4	4. Why Two Critics?	28
10.5	5. Actor Objective	28
10.6	6. How the Actor Gradient Converts Critic Feedback into Weights	29
10.7	7. Role of the Entropy Coefficient in Soft Actor-Critic	30
10.8	8. Soft Update for Target Critics	30
11	Centralized Training and Decentralized Execution (CTDE)	30
12	Task-Level Training Loop	31
12.1	Constructing a True Transition Using the Next Decision State	31
12.2	Summary of the Training Algorithm	32
13	Evaluation and Deployment	33
13.1	Loading the Model and Deterministic Decisions	33
13.2	Matched Comparison Between UAV and No-UAV	33
13.3	Output Metrics	34
14	Notebook and Plot Generation	35
14.1	Primary Expected Plots	37

15 Interpretation of Results	39
16 Limitations and Scope of Implementation Claims	40
17 Conclusion	40

Abstract

This project implements a dynamic vehicular edge-computing environment in which mobile vehicles generate heterogeneous computational tasks, and a decision must be made for each task regarding its processing location and offloading route. The simulation infrastructure includes 10 fixed edge servers, 5 mobile UAVs acting as communication relays, local and edge processing queues, and a set of hard constraints governing feasible routes. UAV positions are updated using K-Means clustering based on the instantaneous vehicle distribution.

The decision-making core of the project is a discrete multi-agent variant of Soft Actor-Critic with a Centralized Training, Decentralized Execution structure. Vehicles are treated conceptually as decision-making agents, while Actor parameters are shared among them. Based on an agent observation, the Actor produces a probability distribution over valid Actions; in contrast, two centralized Critics observe the global State together with the selected Action and estimate its long-term value. Critic training is based on the Bellman target, Actor training combines Q-value and policy entropy, and target stabilization is performed using Target Critics updated through Soft Updates.

In addition to describing the software architecture, this report explains the relationships among project files, the mathematical relations used in the environment and MASAC, the Replay Buffer logic, Action Masking, the Training Loop, and Evaluation. The report contains 30 code screenshots, each documenting a specific part of the architecture.

1 Overall Project Architecture

The software architecture is divided into several independent layers. The data layer reads vehicle and task records from XML. The RL environment layer maintains vehicle states, computation queues, Edge and UAV positions, Observation, Global State, and Reward. The UAV-control layer extracts traffic hotspots. The system-model layer provides the environment with possible Actions, route constraints, and the outcome metrics of each Action. Finally, `src/masac.py` implements the Actor policy, Critics, Target Critics, Replay Buffer, and learning rules.

The active project execution path can be represented conceptually as:

$$\begin{aligned} \text{XML Data} &\rightarrow \text{Environment} \rightarrow \text{Observation/State/Mask} \\ &\rightarrow \text{Actor Action} \rightarrow \text{Environment Transition} \\ &\rightarrow \text{Reward} \rightarrow \text{Replay and MASAC Update.} \end{aligned}$$

The main files and their roles are listed in Table 1.

Table 1: Main files in the active project flow

File	Role
<code>src/xml_loader.py</code>	Streaming ingestion of Vehicle and Task data from XML and maintenance of the record structures.
<code>src/masac_env.py</code>	Decision environment, queues, Observation, Global State, Reward, temporal progression, and integration of project components.
<code>src/uav_control.py</code>	K-Means clustering, assignment of centers to UAVs, and movement logic.
<code>src/system_model.py</code>	Action Space, Action Mask, route topology, and computation of metrics for each Route.
<code>src/masac.py</code>	Actor, two Critics, Target Critics, Replay Buffer, Action selection, and learning updates.
<code>train_masac.py</code>	Complete training loop over temporal Vehicle/Task traces.
<code>evaluate_masac.py</code>	Execution of the trained Policy, scenario comparison, and generation of evaluation CSV files.
<code>diagnose_network.py</code>	Validation of correctness and baseline communication-model behavior before long training runs.
<code>run_phase3.py</code>	Execution of the training and evaluation chain from a single Entry Point.
VEC _ IoT _ MASAC _ Complete _ Colab _ Matched_Final.ipynb	Colab execution, loading outputs, and plotting final results.

2 Input Data and Temporal Alignment

2.1 Vehicle and Task Records

Two immutable data structures are defined in `src/xml_loader.py`. `VehicleRecord` stores time, identifier, coordinates, speed, and vehicle priority. In addition to time and identifier, `TaskRecord` stores the originating Vehicle, data size, cycles per bit, memory requirement, Criticality level, Deadline, and Slack. This design allows the Observation to be constructed directly from real Trace features and prevents different Tasks from being treated as identical in size, computational complexity, or urgency.

The association between a task and its generating agent is established through

`TaskRecord.creator = VehicleRecord.id.`

Consequently, for each Task, the position and state of its corresponding Vehicle are retrieved at the same Timestamp.

```

14  @dataclass(frozen=True)
15  class VehicleRecord:
16      time_s: float
17      id: str
18      x: float
19      y: float
20      speed: float
21      priority: float
22
23
24  @dataclass(frozen=True)
25  class TaskRecord:
26      time_s: float
27      id: str
28      creator: str
29      data_size_mbit: float
30      cycles_per_bit: float
31      required_memory: float
32      criticality: int
33      deadline_s: float
34      slack_s: float
35

```

Figure 1: Structure of Vehicle and Task records in the simulation input.

2.2 Streaming Input and Timestamp Alignment

Instead of loading the entire XML file into memory, `iter_vehicle_timesteps()` and `iter_task_timesteps()` use `ElementTree.iterparse`. Each Timestamp is yielded as an independent group, and after processing, the corresponding Element is cleared with `elem.clear()` to release memory. This behavior is important for large Datasets because memory complexity is bounded by the number of records in the current Timestamp rather than the full file.

During Training and Evaluation, only Tasks whose generating Vehicle exists in the Snapshot at the same time are retained. If the set of Vehicles present at time t is denoted by $\mathcal{V}_t$ and the Tasks at that time by $\mathcal{T}_t$, the valid set is

$$\mathcal{T}_t^{\text{valid}} = \{k \in \mathcal{T}_t \mid \text{creator}(k) \in \mathcal{V}_t\}.$$

This alignment prevents constructing an Observation for a Vehicle that is absent from the Trace.

```
def iter_vehicle_timesteps(path: str) -> Iterator[Tuple[float, Dict[str, VehicleRecord]]]:
    for event, elem in ET.iterparse(path, events=("start", "end")):
        if event != "end" or elem.tag != "timestep":
            continue
        time_s = float(elem.attrib["time"])
        vehicles: Dict[str, VehicleRecord] = {}
        for v in elem.findall("vehicle"):
            vehicles[v.attrib["id"]] = VehicleRecord(
                time_s=time_s,
                id=v.attrib["id"],
                x=float(v.attrib["x"]),
                y=float(v.attrib["y"]),
                speed=float(v.attrib["speed"]),
                priority=float(v.attrib.get("priority", 1.0)),
            )
        elem.clear()
        yield time_s, vehicles

def iter_task_timesteps(path: str) -> Iterator[Tuple[float, Tuple[TaskRecord, ...]]]:
    for event, elem in ET.iterparse(path, events=("start", "end")):
        if event != "end" or elem.tag != "timestep":
            continue
        time_s = float(elem.attrib["time"])
        tasks = []
        for t in elem.findall("task"):
            tasks.append(
                TaskRecord(
                    time_s=time_s,
                    id=t.attrib["id"],
                    creator=t.attrib["creator"],
                    data_size_mbit=float(t.attrib["data_size"]),
                    cycles_per_bit=float(t.attrib["cycles_per_bit"]),
                    required_memory=float(t.attrib["required_memory"]),
                    criticality=int(t.attrib.get("criticality_level", 1)),
                    deadline_s=float(t.attrib.get("deadline", time_s)),
                    slack_s=float(t.attrib.get("slack_time", 0.0)),
                )
            )
        elem.clear()
        yield time_s, tuple(tasks)
```

Figure 2: Streaming data ingestion and generation of temporal Snapshots.

3 VEC Environment and Resource State

3.1 Environment Configuration

The `EnvironmentConfig` class in `src/masac_env.py` stores environment-level parameters. In the standard project configuration, the simulation area is 2000×2000 meters, with 10 Edges and 5 UAVs. Four Reward weights are also stored in this Config so that the Environment can convert the outcome of an Action into a multi-objective Cost.

```

12  @dataclass
13  class EnvironmentConfig:
14      width_m: float = 2000.0
15      height_m: float = 2000.0
16      num_edges: int = 10
17      num_uavs: int = 5
18      dt_s: float = 1.0
19      uav_speed_mps: float = 35.0
20
21      # Reward weights
22      load_penalty: float = 0.40
23      loss_penalty: float = 1.50
24      energy_penalty: float = 0.15
25      deadline_penalty: float = 2.00

```

Figure 3: Environment parameters and Reward weights.

3.2 Edges, UAVs, and Processing Queues

The MASACVECEnv constructor initializes ten fixed Edges in a 5×2 arrangement. At the beginning of each Episode, UAVs are placed at deterministic positions around the center of the area and are subsequently repositioned as traffic moves. A queue measured in CPU cycles is maintained for each Edge, while each Vehicle maintains its own Local queue.

If $Q_e(t)$ is the number of waiting cycles at Edge e at time t and f_e is the CPU frequency of that Edge, queue load in seconds is defined as

$$L_e(t) = \frac{Q_e(t)}{f_e}.$$

The same quantity is used in the Global State and in Load Balancing evaluation.

```

41 def __init__(
42     self, vehicles_by_id: Dict[str, VehicleRecord], edge_positions=None, uav_positions=None, env_cfg: EnvironmentConfig = EnvironmentConfig(), net_cfg: NetworkConfig = NetworkConfig(),
43 ):
44     self.env_cfg = env_cfg
45     self.net = VECSystemModel(net_cfg)
46     self.vehicles_by_id = vehicles_by_id
47     default_edge_positions = [ (200.0, 500.0), (600.0, 500.0), (1000.0, 500.0), (1400.0, 500.0), (1800.0, 500.0), (200.0, 1500.0), (600.0, 1500.0),
48                               (1000.0, 1500.0), # E7
49                               (1400.0, 1500.0), # E8
50                               (1800.0, 1500.0), # E9
51 ]
52     self.edges_xy = list(edge_positions) if edge_positions is not None else default_edge_positions
53     if len(self.edges_xy) != int(self.env_cfg.num_edges):
54         raise ValueError(
55             f"Expected {self.env_cfg.num_edges} edge positions, "
56             f"got {len(self.edges_xy)}"
57         )
58     k = int(self.env_cfg.num_uavs)
59     if k < 0:
60         raise ValueError("num_uavs must be >= 0")
61     if uav_positions is not None:
62         if len(uav_positions) != k:
63             raise ValueError(
64                 f"Expected {k} UAV positions for num_uavs={k}, "
65                 f"got {len(uav_positions)}"
66             )
67         self.uavs_xy = [
68             (float(x), float(y)) for x, y in uav_positions
69         ]
70     elif k == 0:
71         self.uavs_xy = []
72     else:
73         cx = float(self.env_cfg.width_m) / 2.0
74         cy = float(self.env_cfg.height_m) / 2.0
75         radius = 0.20 * min(
76             float(self.env_cfg.width_m),
77             float(self.env_cfg.height_m),
78         )
79         angles = np.linspace(0.0, 2.0 * np.pi, k, endpoint=False)
80         self.uavs_xy = [
81             (
82                 float(np.clip(cx + radius * np.cos(a), 0.0, self.env_cfg.width_m)),
83                 float(np.clip(cy + radius * np.sin(a), 0.0, self.env_cfg.height_m)),
84             )
85             for a in angles
86         ]
87     self.candidates = self.net.build_candidates(
88         self.env_cfg.num_edges, k
89     )
90     self.edge_queue_cycles = np.zeros(self.env_cfg.num_edges, dtype=np.float64)
91     self.local_queue_cycles: Dict[str, float] = {
92         vid: 0.0 for vid in vehicles_by_id
93     }
94     self.active_task_creators = set(vehicles_by_id.keys())
95     self.current_time_s = self.snapshot_time(vehicles_by_id)
96     self.last_metrics: RouteMetrics | None = None
97     self.last_assignment: RouteCandidate | None = None

```

Figure 4: Initialization of Edges, UAVs, Candidates, and processing queues.

3.3 Queue Servicing

As time advances, queues are serviced according to the available processing capacity. Over an interval Δt , the Edge queue is updated as

$$Q_e(t + \Delta t)^- = \max(0, Q_e(t) - f_e \Delta t).$$

If a new Task requiring $C_k D_k$ processing cycles is assigned to Edge e , after the decision we have

$$Q_e(t + \Delta t) = Q_e(t + \Delta t)^- + C_k D_k.$$

An analogous relation is used for the Local queue of each Vehicle, using the local CPU frequency.

```

147     def _serve_queues(self, elapsed_s: float):
148         if elapsed_s <= 0:
149             return
150
151         cpus = np.asarray(self.net.cfg.edge_cpu_hz, dtype=float)
152         for e in range(len(self.edge_queue_cycles)):
153             served = cpus[e % len(cpus)] * elapsed_s
154             self.edge_queue_cycles[e] = max(
155                 0.0, float(self.edge_queue_cycles[e]) - float(served)
156             )
157
158         local_served = float(self.net.cfg.local_cpu_hz) * elapsed_s
159         for vid in list(self.local_queue_cycles):
160             self.local_queue_cycles[vid] = max(
161                 0.0, float(self.local_queue_cycles[vid]) - local_served
162             )

```

Figure 5: Servicing Local and Edge queues using CPU capacity and elapsed time.

4 UAV Control and Repositioning

4.1 Traffic Hotspot Extraction with K-Means

At each Timestamp where at least as many Vehicles as UAVs are present, vehicle positions are passed to K-Means. If $x_i \in \mathbb{R}^2$ is the position of Vehicle i and μ_j is the center of Cluster j , K-Means approximates the following problem:

$$\min_{\{\mu_j\}, \{c_i\}} \sum_{i=1}^{N_v} \|x_i - \mu_{c_i}\|_2^2, \quad c_i \in \{1, \dots, K\}.$$

In the current configuration, $K = 5$. Therefore, five centers are generated as traffic Targets for the UAVs.

```

5     def compute_traffic_centers(vehicle_positions: np.ndarray, num_uavs: int, seed: int) -> np.ndarray:
6         """Return K-Means centers of the current vehicle positions."""
7         model = KMeans(n_clusters=num_uavs, n_init=10, random_state=seed)
8         model.fit(vehicle_positions)
9         return model.cluster_centers_

```

Figure 6: Computation of five traffic centers using K-Means.

4.2 Center Assignment and Bounded Movement

To prevent all UAVs from moving to the same center, a one-to-one Greedy Assignment is performed. If the position of UAV u is p_u , its Target is c_u , and the maximum displacement

in one Step is

$$\Delta_{\max} = v_{\text{UAV}} \Delta t,$$

then, if the UAV can reach the target center within the current step,

$$p_u^{\text{new}} = c_u \quad \text{if } \|c_u - p_u\| \leq \Delta_{\max},$$

otherwise its movement is limited to the maximum allowed displacement:

$$p_u^{\text{new}} = p_u + \frac{c_u - p_u}{\|c_u - p_u\|} \Delta_{\max}.$$

This relation prevents the UAV from unrealistically Teleporting instantaneously.

```

12 def assign_centers_to_uavs(uav_positions: np.ndarray, centers: np.ndarray) -> list[int]:
13     """
14     Greedy one-to-one assignment.
15
16     This prevents every UAV from chasing the same center. For the small
17     number of UAVs in this course simulation, a greedy assignment is enough
18     for Phase 1. Later this can be replaced by Hungarian matching.
19     """
20     remaining = set(range(len(centers)))
21     assignments = []
22
23     for uav_pos in uav_positions:
24         best_index = min(
25             remaining,
26             key=lambda idx: float(np.linalg.norm(uav_pos - centers[idx]))
27         )
28         assignments.append(best_index)
29         remaining.remove(best_index)
30
31     return assignments
32
33
34 def move_uavs_toward_centers(uavs, centers: np.ndarray, cfg) -> None:
35     current_positions = np.array([u.position for u in uavs])
36     assignments = assign_centers_to_uavs(current_positions, centers)
37
38     max_step = cfg.uav_max_speed_mps * cfg.dt_s
39
40     for uav, center_idx in zip(uavs, assignments):
41         target = centers[center_idx]
42         delta = target - uav.position
43         distance = float(np.linalg.norm(delta))
44
45         if distance <= max_step or distance == 0:
46             new_pos = target
47         else:
48             new_pos = uav.position + delta / distance * max_step
49
50     uav.set_position(new_pos)
51

```

Figure 7: One-to-one assignment of centers to UAVs and bounded movement toward each Target.

4.3 Connecting UAV Control to the Environment Timeline

The `advance()` function first computes elapsed time and services queues, then replaces the current Vehicle Snapshot, and finally executes K-Means and UAV movement. Therefore, the network State at each decision depends on time, Mobility, and previous Workloads.

```

164 def advance(self, vehicles_by_id: Dict[str, VehicleRecord]):
165     """Advance mobility, CPU service and UAV positions to a new timestamp."""
166     new_time = self._snapshot_time(vehicles_by_id)
167     elapsed = 0.0
168     if self.current_time_s is not None and new_time is not None:
169         elapsed = max(0.0, new_time - self.current_time_s)
170     self._serve_queues(elapsed)
171
172     self.vehicles_by_id = vehicles_by_id
173     self.current_time_s = new_time
174
175     # Remove vanished vehicle queues; add newly appearing vehicles.
176     live = set(vehicles_by_id)
177     self.local_queue_cycles = {
178         vid: self.local_queue_cycles.get(vid, 0.0) for vid in live
179     }
180
181     # Dynamic cooperative K-Means UAV positioning.
182     # K=0 intentionally skips K-Means and represents the no-UAV case.
183     k = int(self.env_cfg.num_uavs)
184     if k > 0 and len(vehicles_by_id) >= k:
185         pos = np.asarray(
186             [(v.x, v.y) for v in vehicles_by_id.values()], dtype=float
187         )
188         seed = int(round(new_time or 0.0))
189         centers = compute_traffic_centers(pos, k, seed=seed)
190         current = np.asarray(self.uavs_xy, dtype=float)
191         assignment = assign_centers_to_uavs(current, centers)
192         max_step = self.env_cfg.uav_speed_mps * max(elapsed, self.env_cfg.dt_s)
193         updated = []
194         for i, center_idx in enumerate(assignment):
195             delta = centers[center_idx] - current[i]
196             dist = float(np.linalg.norm(delta))
197             if dist == 0.0 or dist <= max_step:
198                 p = centers[center_idx]
199             else:
200                 p = current[i] + delta / dist * max_step
201             updated.append((float(p[0]), float(p[1])))
202         self.uavs_xy = updated

```

Figure 8: Integration of queue servicing and UAV repositioning into the environment Timeline.

5 Action Space and Hard Constraints

5.1 Action-Space Construction

The project uses a discrete Action space composed of four families:

$$\mathcal{A} = \mathcal{A}_{\text{local}} \cup \mathcal{A}_{\text{direct}} \cup \mathcal{A}_{\text{uav}} \cup \mathcal{A}_{\text{wired}}.$$

For 10 Edges and 5 UAVs,

$$|\mathcal{A}_{\text{local}}| = 1, \quad |\mathcal{A}_{\text{direct}}| = 10,$$

$$|\mathcal{A}_{\text{uav}}| = 5 \times 10 = 50.$$

The Edge-neighborhood graph produces 26 valid directed edges for One-Hop routing; therefore,

$$|\mathcal{A}| = 1 + 10 + 50 + 26 = 87.$$

These 87 Actions are created before an Episode begins and retain fixed indices.

```

322 # -----
323 # Route candidates and validity
324 # -----
325 def build_candidates(self, num_edges: int, num_uavs: int):
326     actions = [RouteCandidate(0, "LOCAL", "LOCAL")]
327     idx = 1
328
329     for e in range(num_edges):
330         actions.append(RouteCandidate(idx, "DIRECT", f"DIRECT_E{e}", final_edge=e))
331         idx += 1
332
333     for u in range(num_uavs):
334         for e in range(num_edges):
335             actions.append(
336                 RouteCandidate(
337                     idx,
338                     "UAV_RELAY",
339                     f"UAV{u}->E{e}",
340                     final_edge=e,
341                     uav=u,
342                 )
343             )
344             idx += 1
345
346     for ingress, final in permutations(range(num_edges), 2):
347         if final in self.wired_neighbors.get(ingress, ()):
348             actions.append(
349                 RouteCandidate(
350                     idx,
351                     "WIRED_1HOP",
352                     f"E{ingress}->E{final}",
353                     ingress_edge=ingress,
354                     final_edge=final,
355                 )
356             )
357             idx += 1
358     return actions

```

Figure 9: Construction of LOCAL, DIRECT, UAV-RELAY, and WIRED-1HOP Actions.

5.2 Action Mask as a Hard Constraint

The existence of a Candidate does not imply that it is executable at every moment. For each decision, a Boolean vector

$$m(s) = [m_1, \dots, m_{|\mathcal{A}|}], \quad m_a \in \{0, 1\}$$

is constructed. The primary rules are

$$m_{\text{LOCAL}} = \mathbf{1}[M_k \leq M_{\text{local}}],$$

$$m_{\text{DIRECT},e} = \mathbf{1}[d(v, e) \leq 300],$$

$$m_{\text{UAV},u,e} = \mathbf{1}[d(v, u) \leq 500] \mathbf{1}[d(u, e) \leq 500],$$

and for wired One-Hop routing,

$$m_{\text{WIRED},i,j} = \mathbf{1}[d(v, i) \leq 300] \mathbf{1}[j \in \mathcal{N}(i)].$$

Consequently, an invalid Action never enters the Policy's selection space instead of merely being discouraged through a negative Reward.

```

360 def valid_action_mask(self, vehicle, uavs_xy, edges_xy, candidates, task=None):
361     mask = np.zeros(len(candidates), dtype=bool)
362
363     # Heterogeneous-resource constraint: a task that exceeds the local
364     # device memory cannot be executed locally and must be offloaded.
365     local_ok = True
366     if task is not None:
367         local_ok = float(task.required_memory) <= float(self.cfg.local_memory_capacity)
368     mask[0] = local_ok
369
370     vehicle_xy = (vehicle.x, vehicle.y)
371
372     for c in candidates:
373         if c.kind == "DIRECT":
374             # Vehicle -> edge is legal only when that edge is at most
375             # 300 m away. Distant direct V2E transmission is forbidden.
376             d_edge = self.distance_m(vehicle_xy, edges_xy[c.final_edge])
377             mask[c.index] = d_edge <= self.cfg.direct_edge_radius_m
378
379         elif c.kind == "UAV_RELAY":
380             # Vehicle -> UAV -> edge. Both wireless hops must satisfy
381             # the UAV coverage constraint.
382             du = self.distance_m(vehicle_xy, uavs_xy[c.uav])
383             de = self.distance_m(uavs_xy[c.uav], edges_xy[c.final_edge])
384             mask[c.index] = (
385                 du <= self.cfg.uav_coverage_radius_m
386                 and de <= self.cfg.uav_coverage_radius_m
387             )
388
389         elif c.kind == "WIRED_1HOP":
390             # The first hop is still Vehicle -> ingress edge, so the
391             # same 300 m direct-V2E restriction applies. The second hop
392             # is exactly one wired hop to a direct neighbor.
393             d_ingress = self.distance_m(vehicle_xy, edges_xy[c.ingress_edge])
394             mask[c.index] = (
395                 d_ingress <= self.cfg.direct_edge_radius_m
396                 and c.final_edge in self.wired_neighbors.get(c.ingress_edge, ())
397             )
398
399     # Do NOT re-enable a distant DIRECT route as a fallback. If LOCAL is
400     # infeasible and no legal network route exists, the caller records
401     # NO_FEASIBLE_ROUTE instead of violating the 300 m hard constraint.
402     return mask

```

Figure 10: Applying memory, 300-meter, UAV-coverage, and One-Hop-neighborhood constraints in the Action Mask.

6 Communication Model and Action-Outcome Computation

This section is responsible for converting a selected Action into the metrics required by the Environment for Reward computation and result reporting. The main relations are implemented in `src/system_model.py`. Although the primary focus of this report is the learning agent, the key system-model equations are also presented to complete the Environment flow.

6.1 Link Rate

According to the project model, link rate is computed as

$$R_{i,j} = \eta B \log_2 \left(1 + \frac{S}{N + I} \right),$$

where

$$S = p_{i,j} G_{i,j}.$$

The parameter η is the link's Bandwidth share, B is total Bandwidth, and I is interference associated with the Link type. Packet Loss is implemented as an SINR-based Surrogate because the input system model does not provide an independent Packet-Loss formula.

```

161 def path_loss_db(self, a_xy, b_xy) -> float:
162     """Exact xi_i,j(tau) equation supplied in the project model."""
163     d = self.distance_m(a_xy, b_xy)
164     n = self._path_loss_exponent(*a_xy)
165     fc = max(float(self.cfg.carrier_frequency_hz), 1.0e-12)
166     return float(
167         10.0 * n * math.log10(d)
168         + 20.0 * math.log10(fc)
169         + 10.0 * n * math.log10((4.0 * math.pi) / (3.0e8))
170     )
171
172 def channel_gain(self, a_xy, b_xy) -> float:
173     """G_i,j(tau) represented as linear power gain from xi in dB."""
174     return float(10.0 ** (-self.path_loss_db(a_xy, b_xy) / 10.0))
175
176 # Backward-compatible name used by MASAC observation code.
177 path_gain = channel_gain
178
179 def received_power(self, src_xy, dst_xy, tx_power_w: float) -> float:
180     """S = p_i,j(tau) * G_i,j(tau)."""
181     return float(max(float(tx_power_w), 0.0) * self.channel_gain(src_xy, dst_xy))
182
183 > def effective_noise(self, src_xy, dst_xy) -> float: ...
184
185 # -----
186 # Interference / radio allocation
187 # -----
188 > def radio_allocation(self, connected_count: int) -> float: ...
189
190 > def v2e_interference(...)
191
192 > def e2c_interference(...)
193
194 > def _packet_loss_surrogate(self, sinr: float) -> float:
195     if not np.isfinite(sinr) or sinr <= 0.0: ...
196     sinr_db = 10.0 * math.log10(max(sinr, 1.0e-30))
197     z = (sinr_db - self.cfg.packet_loss_threshold_db) / max(...)
198     # Numerically stable logistic: high SINR -> low loss.
199     if z >= 0: ...
200     ez = math.exp(min(z, 700.0))
201     return float(1.0 / (1.0 + ez))
202
203 > def link_rate(
204     self,
205     src_xy,
206     dst_xy,
207     *,
208 
```

Figure 11: Core channel functions and computation of link rate and the Packet-Loss Surrogate.

6.2 Processing Time and Energy

If Task size in bits is D_k and the number of cycles per bit is C_k , the required number of cycles is

$$W_k = C_k D_k.$$

Accounting for the queue, processing completion time is

$$T_k^{\text{comp}} = \frac{Q + W_k}{f_{\text{core}}}.$$

The processing-energy relation is implemented according to the model supplied with the project as

$$E_k^{\text{exe}} = \zeta f_{\text{core}}^\gamma C_k.$$

Transmission energy is obtained from

$$E_k^{\text{trans}} = p \frac{D_k}{R}.$$

```

280 # -----
281 # Computation / energy
282 # -----
283 @staticmethod
284 def bits(task: TaskRecord) -> float:
285     # The original dataset expresses data_size in MB; convert MB -> bit.
286     return float(task.data_size_mbit) * 8.0e6
287
288 @classmethod
289 def cycles(cls, task: TaskRecord) -> float:
290     # C_v,k * D_v,k total required cycles.
291     return cls.bits(task) * float(task.cycles_per_bit)
292
293 @staticmethod
294 def compute_time_from_cycles(
295     cycles: float,
296     cpu_hz: float,
297     queued_cycles: float = 0.0,
298 ) -> float:
299     return (max(float(queued_cycles), 0.0) + max(float(cycles), 0.0)) / max(
300         float(cpu_hz), 1.0
301     )
302
303 @classmethod
304 def compute_time(cls, task: TaskRecord, cpu_hz: float, queued_cycles: float = 0.0) -> float:
305     return cls.compute_time_from_cycles(cls.cycles(task), cpu_hz, queued_cycles)
306
307 @staticmethod
308 def compute_energy(task: TaskRecord, cpu_hz: float, coeff: float, gamma: float) -> float:
309     """E_exe = zeta * (f_core)^gamma * C_v,k, exactly as supplied.
310
311     C_v,k is the task's cycles-per-bit field. D_v,k is deliberately NOT
312     multiplied into this energy equation; D_v,k remains part of T_exe via
313     the total-cycle quantity C_v,k * D_v,k.
314     """
315     C_vk = max(float(task.cycles_per_bit), 0.0)
316     return float(
317         float(coeff)
318         * (max(float(cpu_hz), 1.0) ** float(gamma))
319         * C_vk
320     )
321

```

Figure 12: Conversion of a Task into computational Workload and computation of execution time and energy.

6.3 Composition of Route Delays

For a direct Route, total delay includes transmission, queueing, and execution:

$$T^{\text{DIRECT}} = T_{v \rightarrow e}^{\text{tx}} + T_e^{\text{queue}} + T_e^{\text{exe}}.$$

For a UAV Relay,

$$T^{\text{UAV}} = T_{v \rightarrow u}^{\text{tx}} + T_{u \rightarrow e}^{\text{tx}} + T_e^{\text{queue}} + T_e^{\text{exe}}.$$

For a wired Hop,

$$T^{\text{WIRED}} = T_{v \rightarrow i}^{\text{tx}} + T_{i \rightarrow j}^{\text{wired}} + T_j^{\text{queue}} + T_j^{\text{exe}}.$$

Local Execution uses no network link, and its Bitrate is therefore deliberately retained as N/A.

7 Construction of Observation and Global State

7.1 Global State for the Centralized Critic

The `_global_features()` function produces information about the vehicle-population distribution, Edge positions, the Load of each Edge, CPU frequencies, and UAV positions. If the Global Feature vector is denoted by g_t , the centralized State is

$$s_t = [o_t, g_t].$$

With 10 Edges and 5 UAVs, Global Features contain 6 statistical vehicle features, 4 features per Edge, and 2 features per UAV; therefore,

$$\dim(g_t) = 6 + 4(10) + 2(5) = 56.$$

```

223 def _global_features(self) -> np.ndarray:
224     vs = list(self.vehicles_by_id.values())
225     if not vs:
226         veh_stats = np.zeros(6, np.float32)
227     else:
228         xy = np.asarray([[v.x, v.y] for v in vs], dtype=np.float32)
229         sp = np.asarray([v.speed for v in vs], dtype=np.float32)
230         veh_stats = np.array(
231             [
232                 xy[:, 0].mean() / self.env_cfg.width_m,
233                 xy[:, 1].mean() / self.env_cfg.height_m,
234                 xy[:, 0].std() / self.env_cfg.width_m,
235                 xy[:, 1].std() / self.env_cfg.height_m,
236                 sp.mean() / 20.0,
237                 sp.std() / 20.0,
238             ],
239             np.float32,
240         )
241
242     edge_stats = []
243     edge_loads = self.edge_load_seconds()
244     for i, (x, y) in enumerate(self.edges_xy):
245         cpu = self.net.cfg.edge_cpu_hz[i % len(self.net.cfg.edge_cpu_hz)]
246         edge_stats.extend(
247             [
248                 x / self.env_cfg.width_m,
249                 y / self.env_cfg.height_m,
250                 min(edge_loads[i] / 10.0, 10.0),
251                 cpu / 1e10,
252             ]
253         )
254
255     uav_stats = []
256     for x, y in self.uavs_xy:
257         uav_stats.extend(
258             [x / self.env_cfg.width_m, y / self.env_cfg.height_m]
259         )
260     return np.asarray(np.r_[veh_stats, edge_stats, uav_stats], dtype=np.float32)

```

Figure 13: Construction of the Global State from vehicle statistics, Edge Load/CPU, and UAV positions.

7.2 Agent Observation

The local Observation consists of 11 Vehicle/Task/Local-Queue features, Vehicle-to-Edge channel Gain for 10 Edges, and two features—Gain and Distance—for each UAV. Therefore, with 5 UAVs,

$$\dim(o_t) = 11 + 10 + 2(5) = 31.$$

Consequently, the State provided to the Critic has dimension

$$\dim(s_t) = 31 + 56 = 87.$$

This number also equals the Action Dimension in the standard configuration, although this equality is merely a consequence of the current parameterization and is not an architectural requirement.

The agent Observation can be written as

$$o_t = [x_v, y_v, v_v, p_v, D_k, C_k, M_k, c_k, \text{slack}_k, \text{deadline}_k, L_v, \text{channel features}],$$

where all components are Normalized or Clipped before entering the network.

```

262 def observation(
263     self, task: TaskRecord, vehicle: VehicleRecord
264 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
265     local_queue_s = self.local_queue_cycles.get(vehicle.id, 0.0) / max(
266         self.net.cfg.local_cpu_hz, 1.0
267     )
268
269     local = np.array(
270         [
271             vehicle.x / self.env_cfg.width_m,
272             vehicle.y / self.env_cfg.height_m,
273             vehicle.speed / 20.0,
274             min(vehicle.priority / 5.0, 1.0),
275             min(task.data_size_mbit / 10.0, 1.0),
276             min(task.cycles_per_bit / 5.0, 1.0),
277             min(task.required_memory / 5000.0, 1.0),
278             min(task.criticality / 5.0, 1.0),
279             min(max(task.slack_s, 0.0) / 20.0, 1.0),
280             min(max(task.deadline_s - task.time_s, 0.0) / 30.0, 1.0),
281             min(local_queue_s / 10.0, 10.0),
282         ],
283         dtype=np.float32,
284     )
285
286     chans = []
287     for exy in self.edges_xy:
288         chans.append(self.net.path_gain((vehicle.x, vehicle.y), exy))
289     for uxy in self.uavs_xy:
290         chans.append(self.net.path_gain((vehicle.x, vehicle.y), uxy))
291         chans.append(
292             self.net.distance_m((vehicle.x, vehicle.y), uxy) / 2500.0
293         )
294
295     obs = np.r_[local, np.asarray(chans, np.float32)]
296     state = np.r_[obs, self._global_features()]
297     mask = self.net.valid_action_mask(
298         vehicle,
299         self.uavs_xy,
300         self.edges_xy,
301         self.candidates,
302         task=task,
303     )
304     return obs.astype(np.float32), state.astype(np.float32), mask

```

Figure 14: Construction of the Observation, concatenation of Global Features, and generation of the Action Mask.

8 Reward and Single-Decision Dynamics

After an Action is selected, `step_task()` first rechecks its validity, computes its Route Metrics, and then adds the Task Workload to the destination queue. Reward is defined as the negative of a multi-objective Cost:

$$r_t = - \left(T_t + \lambda_E E_t + \lambda_P P_t^{\text{loss}} + \lambda_L B_t + \lambda_D M_t^{\text{deadline}} \right).$$

Here, T_t is total latency, E_t is energy, P_t^{loss} is Packet Loss, B_t is the load-imbalance measure, and M_t^{deadline} is the Deadline Miss magnitude.

Load imbalance is computed using the coefficient of variation of Edge queues:

$$B_t = \frac{\sigma(L_1, \dots, L_E)}{\mu(L_1, \dots, L_E)}.$$

If the Deadline Window for a Task is D_k^{max} , the Miss magnitude is

$$M_t^{\text{deadline}} = \max(0, T_t - D_k^{\text{max}}).$$

In the current Config, the coefficients are $\lambda_E = 0.15$, $\lambda_P = 1.50$, $\lambda_L = 0.40$, and $\lambda_D = 2.00$. These coefficients are Hyperparameters; the Reward itself is an Environment output and is not a Hyperparameter.

```

321 def step_task(
322     self, task: TaskRecord, vehicle: VehicleRecord, action: int
323 ) -> Tuple[float, Dict]:
324     mask = self.net.valid_action_mask(
325         vehicle,
326         self.uavs_xy,
327         self.edges_xy,
328         self.candidates,
329         task=task,
330     )
331     if action < 0 or action >= len(mask) or not mask[action]:
332         return -10.0, {"invalid": True, "success": 0.0}
333
334     metric = self.evaluate_action(task, vehicle, action)
335     self.last_metrics = metric
336     self.last_assignment = self.candidates[action]
337
338     cycles = self.net.cycles(task)
339     if metric.final_edge is not None:
340         self.edge_queue_cycles[int(metric.final_edge)] += cycles
341     elif self.candidates[action].kind == "LOCAL":
342         self.local_queue_cycles[vehicle.id] = (
343             self.local_queue_cycles.get(vehicle.id, 0.0) + cycles
344         )
345
346     edge_loads = self.edge_load_seconds()
347     imbalance = self._load_cv()
348
349     deadline_window = float(max(task.deadline_s - task.time_s, 0.0))
350     missed = (
351         float(max(metric.latency_s - deadline_window, 0.0))
352         if deadline_window > 0.0
353         else 0.0
354     )
355
356     reward = -(
357         metric.latency_s
358         + self.env_cfg.energy_penalty * metric.energy_j
359         + self.env_cfg.loss_penalty * metric.packet_loss
360         + self.env_cfg.load_penalty * imbalance
361         + self.env_cfg.deadline_penalty * missed
362     )
363
364     final_edge_load = float("nan")
365     if metric.final_edge is not None:
366         final_edge_load = float(edge_loads[int(metric.final_edge)])
367
368     info = {
369         "latency_s": float(metric.latency_s),
370         "energy_j": float(metric.energy_j),
371         "packet_loss": float(metric.packet_loss),
372         "avg_rate_bps": float(metric.avg_rate_bps),
373         "load_imbalance": float(imbalance),
374         "deadline_miss_s": float(missed),
375         "route": metric.route,
376         "final_edge": metric.final_edge,
377         "final_edge_load": final_edge_load,
378         "success": float(missed <= 1e-12),
379     }
380     for edge_id, load_s in enumerate(edge_loads):
381         info[f"edge{edge_id}_load"] = float(load_s)
382     return float(reward), info
383

```

Figure 15: Execution of an Action, update of the destination queue, and construction of the multi-objective Reward.

9 MASAC Agent Architecture

9.1 Precise Definition of the Algorithm in This Project

The current implementation is a **Discrete MASAC-style CTDE** system with sequential task-level decisions. Vehicles are conceptual Agents, but the Actor is Shared across all Vehicles. Critics observe the global State, while the Action input to a Critic is only the Action of the currently deciding agent; no simultaneous Joint Action is constructed for all Vehicles. Therefore, the precise description of the implementation is: “Shared-policy, task-level sequential multi-agent SAC with centralized critics.”

At each decision, the Actor with parameters θ produces the policy

$$\pi_{\theta}(a \mid o).$$

Two Critics with parameters ϕ_1 and ϕ_2 approximate

$$Q_{\phi_1}(s, a), \quad Q_{\phi_2}(s, a),$$

and two Target Critics with parameters $\bar{\phi}_1, \bar{\phi}_2$ are slowly changing copies of them.

9.2 Hyperparameters and MLP Structure

The primary parameters are $\gamma = 0.98$, $\tau = 0.01$, $\alpha = 0.15$, a Learning Rate of 3×10^{-4} , a Batch Size of 128, and a Hidden Size of 256. The Actor uses the architecture

$$31 \rightarrow 256 \rightarrow 256 \rightarrow 87.$$

The Critic receives $[s, \text{onehot}(a)]$ as input; therefore, in the standard configuration, its input dimension is

$$87 + 87 = 174,$$

and its architecture is

$$174 \rightarrow 256 \rightarrow 256 \rightarrow 1.$$

For an MLP with two hidden layers,

$$h_1 = \text{ReLU}(W_1x + b_1),$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2),$$

$$y = W_3h_2 + b_3.$$

For the Actor, y is the Logit vector; for the Critic, y is a scalar Q-value.

```

20 @dataclass
21 class MASACConfig:
22     gamma: float = 0.98
23     tau: float = 0.01
24     alpha: float = 0.15
25     lr: float = 3e-4
26     hidden: int = 256
27     batch_size: int = 128
28     replay_size: int = 100_000
29     warmup: int = 1_000
30     updates_per_step: int = 1
31     seed: int = 42
32     # Automatic GPU support. Set to 'cpu' to force CPU.
33     device: str = "cuda" if torch.cuda.is_available() else "cpu"
34
35
36 class MLP(nn.Module):
37     def __init__(self, inp: int, hidden: int, out: int):
38         super().__init__()
39         self.net = nn.Sequential(
40             nn.Linear(inp, hidden), nn.ReLU(),
41             nn.Linear(hidden, hidden), nn.ReLU(),
42             nn.Linear(hidden, out),
43         )
44
45     def forward(self, x):
46         return self.net(x)
47

```

Figure 16: MASAC Hyperparameters and the Actor/Critic neural-network structure.

9.3 Replay Buffer

Each Interaction is stored as the following Transition:

$$\tau_t = (s_t, a_t, r_t, s_{t+1}, d_t, m_t, m_{t+1}).$$

Here, d_t is the Terminal Flag and m_t, m_{t+1} are the Action Masks. The Replay Buffer forms the set $\mathcal{D}$, and a training Batch is sampled as

$$\mathcal{B} \sim \text{Uniform}(\mathcal{D}).$$

This reduces strong temporal dependence between consecutive Trace samples and allows an experience to be reused in multiple stochastic Updates.

```

49 class ReplayBuffer:
50     def __init__(self, size: int):
51         self.size = size
52         self.data = []
53         self.ptr = 0
54
55     def add(self, s, a, r, ns, done, mask, next_mask):
56         item = (
57             np.asarray(s, np.float32), int(a), float(r),
58             np.asarray(ns, np.float32), float(done),
59             np.asarray(mask, bool), np.asarray(next_mask, bool)
60         )
61         if len(self.data) < self.size:
62             self.data.append(item)
63         else:
64             self.data[self.ptr] = item
65             self.ptr = (self.ptr + 1) % self.size
66
67     def __len__(self):
68         return len(self.data)
69
70     def sample(self, batch: int, device: torch.device):
71         items = random.sample(self.data, batch)
72         s, a, r, ns, d, m, nm = zip(*items)
73         return (
74             torch.as_tensor(np.stack(s), dtype=torch.float32, device=device),
75             torch.as_tensor(a, dtype=torch.long, device=device),
76             torch.as_tensor(r, dtype=torch.float32, device=device),
77             torch.as_tensor(np.stack(ns), dtype=torch.float32, device=device),
78             torch.as_tensor(d, dtype=torch.float32, device=device),
79             torch.as_tensor(np.stack(m), dtype=torch.bool, device=device),
80             torch.as_tensor(np.stack(nm), dtype=torch.bool, device=device),
81         )
82

```

Figure 17: Storage of Transitions and random Minibatch sampling for training.

9.4 Actor, Critics, and Target Critics

The DiscreteMASAC constructor creates one Actor, two primary Critics, and two Target Critics. At the beginning of training, the Targets are copied exactly from the primary Critics:

$$\bar{\phi}_1 \leftarrow \phi_1, \quad \bar{\phi}_2 \leftarrow \phi_2.$$

The Critics have independent Optimizers, and the Actor is updated separately with Adam. Target Critics do not have direct Optimizers and are updated only through Soft Updates.

```

84 class DiscreteMASAC:
85     def __init__(self, obs_dim: int, state_dim: int, action_dim: int, cfg: MASACConfig = MASACConfig()):
86         self.cfg = cfg
87         random.seed(cfg.seed)
88         np.random.seed(cfg.seed)
89         torch.manual_seed(cfg.seed)
90         if torch.cuda.is_available():
91             torch.cuda.manual_seed_all(cfg.seed)
92
93         requested = str(cfg.device)
94         if requested.startswith("cuda") and not torch.cuda.is_available():
95             requested = "cpu"
96         self.device = torch.device(requested)
97         self.obs_dim = int(obs_dim)
98         self.action_dim = int(action_dim)
99
100        self.actor = MLP(obs_dim, cfg.hidden, action_dim).to(self.device)
101        critic_in = state_dim + action_dim
102        self.q1 = MLP(critic_in, cfg.hidden, 1).to(self.device)
103        self.q2 = MLP(critic_in, cfg.hidden, 1).to(self.device)
104        self.tq1 = MLP(critic_in, cfg.hidden, 1).to(self.device)
105        self.tq2 = MLP(critic_in, cfg.hidden, 1).to(self.device)
106        self.tq1.load_state_dict(self.q1.state_dict())
107        self.tq2.load_state_dict(self.q2.state_dict())
108
109        self.oa = torch.optim.Adam(self.actor.parameters(), lr=cfg.lr)
110        self.o1 = torch.optim.Adam(self.q1.parameters(), lr=cfg.lr)
111        self.o2 = torch.optim.Adam(self.q2.parameters(), lr=cfg.lr)
112        self.replay = ReplayBuffer(cfg.replay_size)

```

Figure 18: Construction of the Actor, Q1/Q2, Target Q1/Q2, and independent Optimizers.

9.5 Policy Masking and Action Selection

The Actor first produces the Logit vector

$$z_{\theta}(o) = [z_1, \dots, z_{|\mathcal{A}|}].$$

The Mask then maps invalid Logits to a very negative value. For a valid Action ($m_a = 1$), the Logit remains unchanged:

$$\tilde{z}_a = z_a,$$

while for an invalid Action ($m_a = 0$) it is mapped toward a very negative value:

$$\tilde{z}_a \rightarrow -\infty.$$

In the implementation, the finite value $-1\text{e}9$ is used as a numerical approximation of $-\infty$. The discrete Policy is obtained by Softmax:

$$\pi_{\theta}(a \mid o) = \frac{e^{\tilde{z}_a}}{\sum_b e^{\tilde{z}_b}}.$$

Therefore, for an invalid Action, $\pi_{\theta}(a \mid o) \approx 0$.

During Training, an Action is sampled stochastically from the Categorical Policy:

$$a_t \sim \pi_\theta(\cdot \mid o_t),$$

whereas during Evaluation the selection is Deterministic:

$$a_t^* = \arg \max_{a: m_a=1} \pi_\theta(a \mid o_t).$$

```

114     @staticmethod
115     def masked_logits(logits, mask):
116         mask = mask.bool()
117         # The environment should always leave at least LOCAL valid.
118         return logits.masked_fill(~mask, -1e9)
119
120     @torch.no_grad()
121     def act(self, obs: np.ndarray, mask: np.ndarray, deterministic: bool = False) -> int:
122         x = torch.as_tensor(obs, dtype=torch.float32, device=self.device).unsqueeze(0)
123         m = torch.as_tensor(mask, dtype=torch.bool, device=self.device).unsqueeze(0)
124         logits = self.masked_logits(self.actor(x), m)
125         if deterministic:
126             return int(logits.argmax(-1).item())
127         return int(Categorical(logits=logits).sample().item())
128

```

Figure 19: Application of the Action Mask and the difference between Sampling in Training and Argmax in Evaluation.

9.6 Construction of Probability and Log-Probability

The `_dist()` function uses a Categorical Distribution to return $\pi(a \mid o)$ and $\log \pi(a \mid o)$ for all Actions. These quantities are used directly in the Entropy Term and Expected Soft Value.

```

129     def _dist(self, obs, mask):
130         logits = self.masked_logits(self.actor(obs), mask)
131         dist = Categorical(logits=logits)
132         probs = dist.probs
133         logp = torch.log(probs.clamp_min(1e-8))
134         return probs, logp
135
136     def update(self):
137         if len(self.replay) < max(self.cfg.batch_size, self.cfg.warmup):
138             return None
139
140         stats = None
141         for _ in range(self.cfg.updates_per_step):
142             s, a, r, ns, d, mask, next_mask = self.replay.sample(self.cfg.batch_size, self.device)
143             onehot = torch.nn.functional.one_hot(a, self.action_dim).float()
144             q1 = self.q1(torch.cat([s, onehot], dim=-1)).squeeze(-1)
145             q2 = self.q2(torch.cat([s, onehot], dim=-1)).squeeze(-1)
146

```

Figure 20: Generation of Probability/Log-Probability and computation of Q for the actual Batch Action.

10 Mathematical Training of MASAC

This is the central learning section. An Update is performed once the Replay Buffer reaches

$$|\mathcal{D}| \geq \max(\text{batch size}, \text{warmup}).$$

In the current Config, Warm-up is 1000.

10.1 1. Critic Prediction for the Actual Action

For each sample (s_t, a_t) in the Batch, the Action is converted to One-Hot form:

$$e(a_t) \in \{0, 1\}^{|A|}.$$

The Critic input is

$$x_t^Q = [s_t, e(a_t)],$$

and the two predictions

$$q_{1,t} = Q_{\phi_1}(s_t, a_t), \quad q_{2,t} = Q_{\phi_2}(s_t, a_t)$$

are computed.

10.2 2. Soft Value of the Next State

For the next State, the Actor produces probabilities over all valid Actions:

$$\pi_\theta(a' \mid o_{t+1}).$$

For each Action, the Target Critics produce

$$Q_{\bar{\phi}_1}(s_{t+1}, a'), \quad Q_{\bar{\phi}_2}(s_{t+1}, a'),$$

and the conservative estimate is

$$Q_{\text{targ}}(s', a') = \min \left(Q_{\bar{\phi}_1}(s', a'), Q_{\bar{\phi}_2}(s', a') \right).$$

The Soft State Value is then defined as an expectation over all discrete Actions:

$$V_{\text{soft}}(s') = \sum_{a'} \pi_\theta(a' \mid o') [Q_{\text{targ}}(s', a') - \alpha \log \pi_\theta(a' \mid o')].$$

The first term represents exploitation, while the second preserves Policy Entropy.

```

147         with torch.no_grad():
148             p2, logp2 = self._dist(ns[:, :self.obs_dim], next_mask)
149             vals = []
150             for ai in range(self.action_dim):
151                 aidx = torch.full((ns.size(0),), ai, dtype=torch.long, device=self.device)
152                 oh = torch.nn.functional.one_hot(aidx, self.action_dim).float()
153                 qa = torch.min(
154                     self.tq1(torch.cat([ns, oh], -1)).squeeze(-1),
155                     self.tq2(torch.cat([ns, oh], -1)).squeeze(-1),
156                 )
157                 # Invalid next actions are masked by p2=0.
158                 vals.append(qa)
159             qmat = torch.stack(vals, dim=1)
160             v = torch.sum(p2 * (qmat - self.cfg.alpha * logp2), dim=1)
161             target = r + self.cfg.gamma * (1.0 - d) * v

```

Figure 21: Computation of the Soft Value of the next State and the Bellman Target using the Target Critics.

10.3 3. Bellman Target and Critic Training

The Bellman Target is

$$y_t = r_t + \gamma(1 - d_t)V_{\text{soft}}(s_{t+1}).$$

If the Transition is Terminal, $d_t = 1$ and the Future Value is removed. Each Critic is trained using the Mean Squared Bellman Error:

$$L_{Q_1}(\phi_1) = \mathbb{E}_{\mathcal{B}} \left[(Q_{\phi_1}(s, a) - y)^2 \right]$$

and

$$L_{Q_2}(\phi_2) = \mathbb{E}_{\mathcal{B}} \left[(Q_{\phi_2}(s, a) - y)^2 \right].$$

Gradient Descent updates the Critic parameters as

$$\phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} L_{Q_i}.$$

Thus, the Critic is trained directly, while the Target Critic is used only to construct a stable target.

```

163         lq1 = ((q1 - target) ** 2).mean()
164         lq2 = ((q2 - target) ** 2).mean()
165         self.o1.zero_grad(set_to_none=True)
166         lq1.backward()
167         self.o1.step()
168         self.o2.zero_grad(set_to_none=True)
169         lq2.backward()
170         self.o2.step()

```

Figure 22: Computation of Bellman Error and direct training of Q1 and Q2 through Backpropagation.

10.4 4. Why Two Critics?

Using a single Critic can lead to Overestimation. Therefore, both the Target and Actor Update use

$$Q_{\min}(s, a) = \min(Q_{\phi_1}(s, a), Q_{\phi_2}(s, a)).$$

For example, if

$$Q_1 = -2.0, \quad Q_2 = -2.7,$$

the value used is -2.7 . This conservative choice reduces the probability that the Policy will learn from overly optimistic estimates.

10.5 5. Actor Objective

After the Critic Updates, the Actor produces probabilities over all valid Actions for the Batch States. The Actor aims to increase Expected Q while simultaneously preserving Entropy. The discrete Loss used in the code is

$$L_{\pi}(\theta) = \mathbb{E}_{s \sim \mathcal{B}} \left[\sum_a \pi_{\theta}(a | o) (\alpha \log \pi_{\theta}(a | o) - Q_{\min}(s, a)) \right].$$

Since

$$\mathcal{H}(\pi(\cdot | o)) = - \sum_a \pi(a | o) \log \pi(a | o),$$

minimizing the Loss above is approximately equivalent to maximizing

$$\mathbb{E}[Q_{\min}(s, a)] + \alpha \mathcal{H}(\pi(\cdot | o)).$$

The Actor therefore has two simultaneous objectives: selecting Actions for which the Critic predicts higher Long-Term Return, and preventing premature collapse of the Policy to a fully Deterministic Action.

```

172 p, logp = self._dist(s[:, :self.obs_dim], mask)
173 vals = []
174 for ai in range(self.action_dim):
175     aidx = torch.full((s.size(0),), ai, dtype=torch.long, device=self.device)
176     oh = torch.nn.functional.one_hot(aidx, self.action_dim).float()
177     qa = torch.min(
178         self.q1(torch.cat([s, oh], -1)).squeeze(-1),
179         self.q2(torch.cat([s, oh], -1)).squeeze(-1),
180     )
181     vals.append(qa)
182 qmat = torch.stack(vals, dim=1)
183 actor_loss = torch.sum(p * (self.cfg.alpha * logp - qmat), dim=1).mean()
184 self.oa.zero_grad(set_to_none=True)
185 actor_loss.backward()
186 self.oa.step()
187

```

Figure 23: Computation of Actor Loss from Q1/Q2 and Entropy, followed by updating Actor parameters.

10.6 6. How the Actor Gradient Converts Critic Feedback into Weights

The Actor is not modified directly by the Critic. Q-values enter the Loss, after which Back-propagation computes the Gradient with respect to Actor parameters:

$$\theta \leftarrow \theta - \eta_{\pi} \nabla_{\theta} L_{\pi}(\theta).$$

Using

$$\nabla_{\theta} \pi_{\theta}(a | o) = \pi_{\theta}(a | o) \nabla_{\theta} \log \pi_{\theta}(a | o),$$

the Gradient can be written as

$$\nabla_{\theta} L_{\pi} = \mathbb{E} \left[\sum_a \pi_{\theta}(a | o) \nabla_{\theta} \log \pi_{\theta}(a | o) (\alpha \log \pi_{\theta}(a | o) - Q_{\min}(s, a) + \alpha) \right].$$

In practice, PyTorch computes this derivative with `actor_loss.backward()`, and Adam updates the weights through `self.oa.step()`. Actions with better Q values tend to receive higher Probability, while the Entropy Term prevents the Probabilities of other valid Actions from rapidly collapsing to zero.

10.7 7. Role of the Entropy Coefficient in Soft Actor-Critic

α controls the strength of entropy Regularization. As $\alpha \rightarrow 0$, Actor Loss approaches pure Q maximization and the Policy becomes more Greedy. Increasing α produces a flatter Probability distribution and more Exploration. In the current implementation, $\alpha = 0.15$ is fixed and Automatic Entropy Tuning is not implemented.

10.8 8. Soft Update for Target Critics

Target Critics are not trained directly with Gradient Descent. After each Update, their parameters are changed through Polyak Averaging:

$$\bar{\phi}_i \leftarrow (1 - \tau)\bar{\phi}_i + \tau\phi_i, \quad i \in \{1, 2\}.$$

With $\tau = 0.01$, only one percent of each primary Critic update is incorporated into its Target at each step. Consequently, the Bellman target changes more slowly than it would if the actively trained network were used directly, improving Training stability.

```

188         with torch.no_grad():
189             for tgt, src in ((self.tq1, self.q1), (self.tq2, self.q2)):
190                 for tp, sp in zip(tgt.parameters(), src.parameters()):
191                     tp.mul_(1.0 - self.cfg.tau).add_(self.cfg.tau * sp)
192             stats = {
193                 "q_loss": float((lq1 + lq2).item()),
194                 "actor_loss": float(actor_loss.item()),
195             }
196         return stats

```

Figure 24: Slow update of Target Critics using coefficient τ .

11 Centralized Training and Decentralized Execution (CTDE)

During Training, the Actor receives only the Observation o_i of the agent currently making a decision, while the Critic has access to the global State s :

$$\text{Actor: } \pi_{\theta}(a_i | o_i), \quad \text{Critic: } Q_{\phi}(s, a_i).$$

This information asymmetry allows the Critic to better evaluate the effect of a decision on queues, the overall Edge state, and UAV positions. After Training, the Critic is no longer

required for Action selection, and the Policy executes

$$a_i^* = \arg \max_{a:m_a=1} \pi_\theta(a \mid o_i).$$

Thus, during Deployment the decision path consists of the Actor, Observation, and Action Mask; Critics are learning instruments rather than components of Policy execution.

12 Task-Level Training Loop

12.1 Constructing a True Transition Using the Next Decision State

In `train_masac.py`, each Episode advances over temporal Snapshots. A key detail is that the previous Transition is retained in the variable `pending` until the next Decision State arrives. Once the next Task Observation has been constructed, that State is inserted into the previous Transition as s_{t+1} before the Transition enters Replay. Therefore, Replay does not use an artificial immediate State; Next State is the actual next decision State in the Trace.

The flow of a Transition is

$$(s_t, a_t, r_t) \xrightarrow{\text{next task arrives}} (s_t, a_t, r_t, s_{t+1}) \rightarrow \mathcal{D}.$$

For the final Task of an Episode, the Transition is recorded as Terminal with $d = 1$, and Future Value is removed from the Bellman Target.

```

108 for ep in range(args.episodes):
109     env = MASACEnv(envs[0], env_cfg=env_cfg)
110     recent = []
111     pending = None
112     for t, tasks in groups:
113         env.advance(vehicles[t])
114         env.set_active_task_creators(task_creator for task in tasks)
115         for task in tasks:
116             vehicle = vehicles[t][task.creator]
117             obs, state, mask = env.observation(task, vehicle)
118             # Complete the preceding replay transition with the actual
119             # next decision state now that it is available.
120             if pending is not None:
121                 p_state, p_action, p_reward, p_mask = pending
122                 agent.replay.add(
123                     p_state,
124                     p_action,
125                     p_reward,
126                     state,
127                     False,
128                     p_mask,
129                     mask,
130                 )
131             agent.update()
132             # The 100s hard restriction can legitimately leave a
133             # memory-constrained task with no feasible route. Do not let
134             # the policy silently choose a masked distant DIRECT action.
135             if not mask.any():
136                 if pending is not None:
137                     p_state, p_action, p_reward, p_mask = pending
138                     agent.replay.add(
139                         p_state,
140                         p_action,
141                         p_reward,
142                         state,
143                         True,
144                         p_mask,
145                         mask,
146                     )
147                 agent.update()
148                 pending = None
149             recent.append({
150                 "latency_s": float("nan"),
151                 "energy_j": 0.0,
152                 "packet_loss": 1.0,
153                 "avg_rate_bps": float("nan"),
154                 "route": "NO_FEASIBLE_ROUTE",
155                 "final_edges": None,
156                 "success": 0.0,
157             })
158             continue
159             action = agent.act(obs, mask, deterministic=False)
160             reward, info = env.step_task(task, vehicle, action)
161             recent.append(info)
162             pending = (state, action, reward, mask)
163             # Terminal transition for the last task of the episode.
164             if pending is not None:
165                 p_state, p_action, p_reward, p_mask = pending
166                 agent.replay.add(
167                     p_state,
168                     p_action,
169                     p_reward,
170                     state,
171                     True,
172                     p_mask,
173                     mask,
174                 )
175             agent.update()

```

Figure 25: Task-level Training loop and connection of the true next State to the Replay Transition.

12.2 Summary of the Training Algorithm

The Training process can be summarized as follows:

1. Receive the current Vehicle and Task Snapshot.
2. Construct Observation o_t , State s_t , and Mask m_t .
3. If the previous Transition is Pending, store s_t as its Next State in Replay.
4. Sample $a_t \sim \pi_\theta(\cdot | o_t)$ from the Policy subject to the Action Mask.
5. Execute the Action in the Environment and obtain Reward r_t .
6. Keep the Transition Pending until the next State arrives.
7. Once Replay has reached Warm-up, sample a random Minibatch.
8. Construct the Bellman Target using the Target Critics and update Q1/Q2.
9. Update the Actor using L_π .
10. Move the Target Critics through a Soft Update.
11. Repeat for subsequent Tasks and Episodes.

13 Evaluation and Deployment

13.1 Loading the Model and Deterministic Decisions

During Evaluation, the same Observation/State/Action dimensions are constructed, the trained Checkpoint is loaded, and the Actor is executed with `deterministic=True`. Exploration is therefore removed and the highest valid Logit is selected.

13.2 Matched Comparison Between UAV and No-UAV

To analyze the effect of UAVs, before executing the Actor’s actual Action, Evaluation examines all valid Routes from the same Pre-action State. Two sets are constructed:

$$\mathcal{R}_{\text{UAV}} = \{a \mid a \text{ is feasible UAV_RELAY}\},$$

$$\mathcal{R}_{\text{NoUAV}} = \{a \mid a \text{ is feasible DIRECT or WIRED_1HOP}\}.$$

LOCAL is deliberately excluded from this communication comparison. A Task enters the comparison Pair only when both sets are non-empty, and the minimum-Latency Route is selected within each scenario. Consequently, the UAV and No-UAV columns are compared on exactly the same Tasks. This component is a scenario Benchmark and is distinct from the actual MASAC Action.

The Actor’s actual Action is subsequently executed as

$$a^* = \arg \max_a \pi_\theta(a \mid o),$$

and its Metrics are recorded in `evaluation.csv`.

```

155 for task in tasks:
156     vehicle = vehicles[t][task.creator]
157     obs, _, mask = env.observation(task, vehicle)
158
159     # -----
160     # Required scenario comparison on the SAME pre-action state.
161     # UAV-relay: feasible UAV-RELAY routes.
162     # No-UAV: feasible DIRECT / WIRED_IHOP routes only.
163     # LOCAL is intentionally excluded from this communication
164     # scenario comparison.
165     # -----
166     uav_options = []
167     no_uav_options = []
168     for c in env.candidates:
169         if not mask[c.index]:
170             continue
171         if c.kind not in ["UAV_RELAY", "DIRECT", "WIRED_IHOP"]:
172             continue
173         metric = env.evaluate_action(task, vehicle, c.index)
174         row = scenario_record(metric, task, vehicle, t, c)
175         if c.kind == "UAV_RELAY":
176             uav_options.append(row)
177         else:
178             no_uav_options.append(row)
179
180     # Keep ONLY matched tasks so the mandatory UAV-vs-no-UAV
181     # comparison is apples-to-apples. A task contributes two rows
182     # (one per scenario) only when BOTH scenario families are feasible
183     # in the exact same pre-action state.
184     if uav_options and no_uav_options:
185         pair_id = f"({task.id},{vehicle.id})"
186         best_uav = min(uav_options, key=lambda r: r["latency_s"])
187         best_no_uav = min(no_uav_options, key=lambda r: r["latency_s"])
188         best_uav["pair_id"] = pair_id
189         best_no_uav["pair_id"] = pair_id
190         best_uav["matched"] = 1
191         best_no_uav["matched"] = 1
192         scenario_rows.extend([best_uav, best_no_uav])
193
194     # -----
195     # Actual trained MASAC decision.
196     # -----
197     if not mask.any():
198         # No legal route exists under LOCAL resource constraints,
199         # the 300 = direct-V2X limit, UAV coverage, and one-hop
200         # wired adjacency. Record a failed task instead of silently
201         # violating the hard 300 m rule.
202         info = {
203             "latency_s": float("nan"),
204             "energy_j": 0.0,
205             "packet_loss": 1.0,
206             "avg_rx_tx_rate": float("nan"),
207             "load_imbalance": float("nan"),
208             "deadline_miss_s": float("nan"),
209             "route": "NO_FEASIBLE_ROUTE",
210             "final_edge": None,
211             "final_edge_load": float("nan"),
212             "success": 0.0,
213             "time_s": t,
214             "task": task.id,
215             "agent": vehicle.id,
216         }
217     else:
218         action = agent.act(obs, mask, deterministic=True)
219         _, info = env.step_task(task, vehicle, action)
220         info = dict(info)
221         info.update({"time_s": t, "task": task.id, "agent": vehicle.id})
222
223     selected_rows.append(info)

```

Figure 26: Matched UAV/No-UAV comparison followed by execution of the Actor’s actual decision during Evaluation.

13.3 Output Metrics

Evaluation records the following metrics:

- Total Latency for each Task;
- Total Energy;
- Packet Loss and Average Network Bitrate only for network Routes;
- Task Success Rate based on Deadline;
- actual Load of each Edge after every Task;
- UAV positions and the number of UAVs present at each Timestamp;
- distribution of Actor decisions.

LOCAL Bitrate is N/A because no communication link is used and is therefore excluded from Average Network Rate. Load is taken from the real queue after the Task, not from the zero Snapshot before the Workload is assigned.

```

238 network_rows = [
239     r for r in selected_rows
240     if np.isfinite(float(r.get("avg_rate_bps", float("nan"))))]
241 ]
242 uav_scn = [r for r in scenario_rows if r["scenario"] == "UAV-relay"]
243 no_uav_scn = [r for r in scenario_rows if r["scenario"] == "No-UAV-relay"]
244
245 route_counts = defaultdict(int)
246 for r in selected_rows:
247     route_counts[r.get("route", "UNKNOWN")] += 1
248
249 print("===== MASAC EVALUATION =====")
250 print(f"Tasks = {len(selected_rows)}")
251 print(f"Average total latency = {finite_mean(selected_rows, 'latency_s'):.6f} s")
252 print(f"Average total energy = {finite_mean(selected_rows, 'energy_j'):.8f} J")
253 if network_rows:
254     print(f"Network packet loss = {finite_mean(network_rows, 'packet_loss'):.6f}")
255     print(f"Average NETWORK data rate = {finite_mean(network_rows, 'avg_rate_bps')/1e6:.3f} Mbps")
256     print(f"Network/offloaded tasks = {len(network_rows)}")
257 else:
258     print(f"Network packet loss = N/A (no network routes selected)")
259     print(f"Average NETWORK data rate = N/A (no network routes selected)")
260     print(f"Network/offloaded tasks = 0")
261 print(f"Task success rate = {100*np.mean([float(r.get('success',0)) for r in selected_rows]):.2f}%")
262 no_routes_count = sum(1 for r in selected_rows if r.get("route") == "NO_FEASIBLE_ROUTE")
263 print(f"No-feasible-route tasks = {no_routes_count}")
264 print("===== REQUIRED MATCHED communication scenario comparison =====")
265 print(f"Matched task pairs = {len(uav_scn)}")
266 print(f"UAV-relay samples = {len(uav_scn)}")
267 print(f"No-UAV samples = {len(no_uav_scn)}")
268 if len(uav_scn) != len(no_uav_scn):
269     raise RuntimeError("Internal error: matched scenario counts are unequal.")
270 if uav_scn:
271     print(f"UAV-relay avg latency = {finite_mean(uav_scn, 'latency_s'):.6f} s")
272     print(f"UAV-relay avg bitrate = {finite_mean(uav_scn, 'avg_rate_bps')/1e6:.3f} Mbps")
273 if no_uav_scn:
274     print(f"No-UAV avg latency = {finite_mean(no_uav_scn, 'latency_s'):.6f} s")
275     print(f"No-UAV avg bitrate = {finite_mean(no_uav_scn, 'avg_rate_bps')/1e6:.3f} Mbps")
276 print("===== MASAC route distribution =====")
277 for route, count in sorted(route_counts.items()):
278     print(f"({route:20s}) = {count}")
279 # Average edge load is queued processing work (seconds) after each task.
280 print(f"===== Average edge load (queued processing work) =====")
281 for edge_id in range(env.env_cfg.num_edges):
282     vals = [r["load"] for r in edge_rows if r["edge"] == edge_id]
283     print(f"Edge {edge_id}: {float(np.mean(vals)) if vals else 0.0:.6f} s")
284 avg_presence = float(np.mean([r["uav_count"] for r in presence_rows])) if presence_rows else 0.0
285 print(f"===== Average UAV presence on map = {avg_presence:.3f} UAVs/timestep =====")
286 write_csv(args.csv, selected_rows)
287 write_csv(
288     args.scenario_csv,
289     scenario_rows,
290     fields=[
291         "time_s", "task", "agent", "scenario", "action_kind", "route",
292         "latency_s", "avg_latency_s", "energy_j", "packet_loss",
293         "avg_rate_bps", "final_energy", "pair_id", "matched",
294     ],
295 )
296 write_csv(args.edge_csv, edge_rows)
297 write_csv(
298     args.uav_csv,
299     uav_rows,
300     fields=[
301         "time_s", "uav_id", "x", "y",
302     ],
303 )
304 write_csv(args.uav_presence_csv, presence_rows)
305 print("===== Generated =====")
306 for path in [
307     args.csv,
308     args.scenario_csv,
309     args.edge_csv,
310     args.uav_csv,
311     args.uav_presence_csv,
312 ]:

```

Figure 27: Computation of Summary Metrics and generation of final evaluation CSV files.

14 Notebook and Plot Generation

The final Notebook, `VEC_IoT_MASAC_Complete_Colab_Matched_Final.ipynb`, provides the execution chain in Colab: mounting Drive, checking Dependencies, running Diagnostics, Training, Evaluation, reading CSV outputs, and plotting results. To avoid Binary incompatibilities, the Notebook does not force a NumPy Upgrade at startup.

```

> /python evaluate_masac.py --vehicles datasets/vehicles.xml --tasks datasets/tasks.xml --model outputs/masac.pt --max-tasks 10000

Loaded MASAC model on cpu
Number of UAVs (K): 5
Number of edge servers: 10

===== MASAC EVALUATION =====
Tasks = 10000
Average total latency = 20.125029 s
Average total energy = 9.82434664 J
Network packet loss = 0.593511
Average NETWORK data rate = 0.683 Mbps
Network/offloaded tasks = 6095
Task success rate = 58.76%
No-feasible-route tasks = 192

Required MATCHED communication scenario comparison:
Matched task pairs = 5719
UAV-relay samples = 5719
No-UAV samples = 5719
UAV-relay avg latency = 37.262259 s
UAV-relay avg bitrate = 0.626 Mbps
No-UAV avg latency = 31.593702 s
No-UAV avg bitrate = 0.753 Mbps

MASAC route distribution:
DIRECT_E0 = 389
...
outputs/evaluation_scenarios.csv
outputs/evaluation_edge_load.csv
outputs/uav_positions.csv
outputs/uav_presence.csv

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings..

```

Figure 28: Loading Evaluation outputs for plotting final results.

14.1 Primary Expected Plots

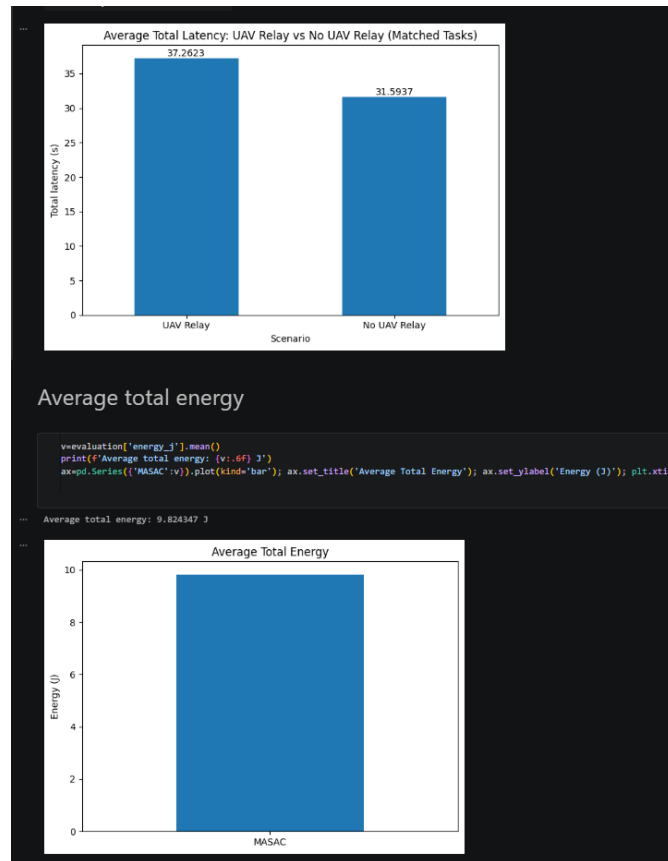


Figure 29: Matched comparison of latency in UAV and No-UAV modes and mean energy consumption

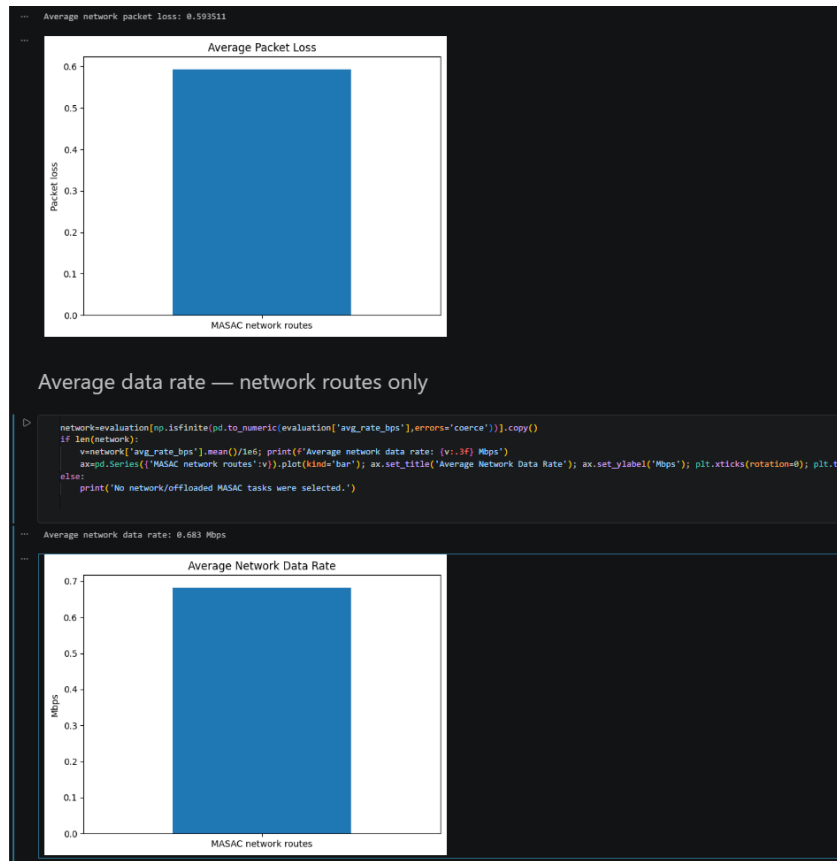


Figure 30: Set of Packet Loss and Bitrate plots

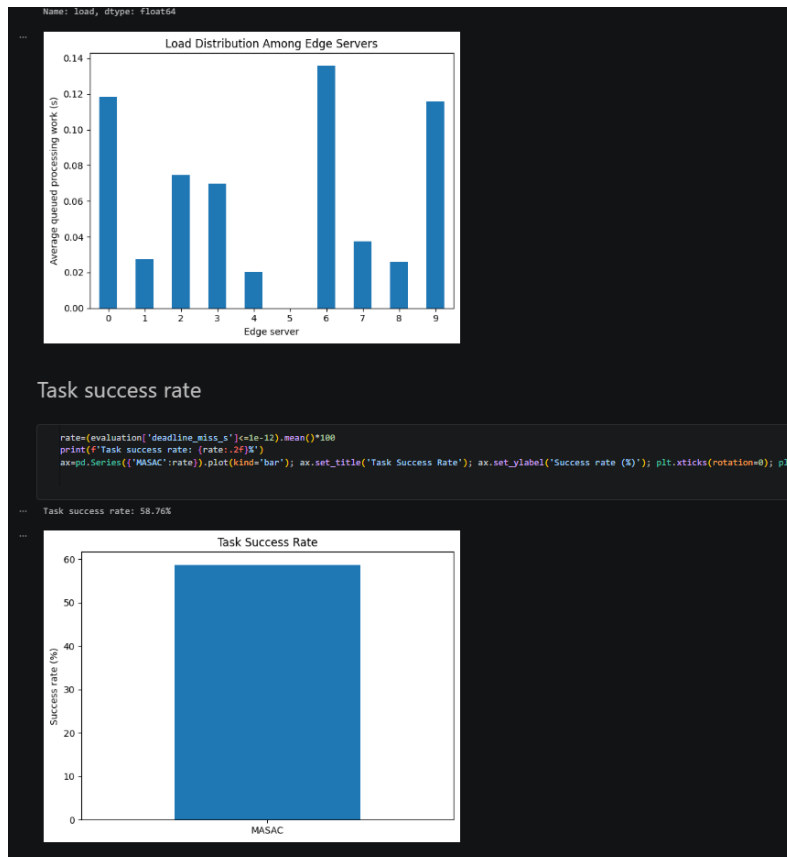


Figure 31: Set of Edge Load and Success Rate plots

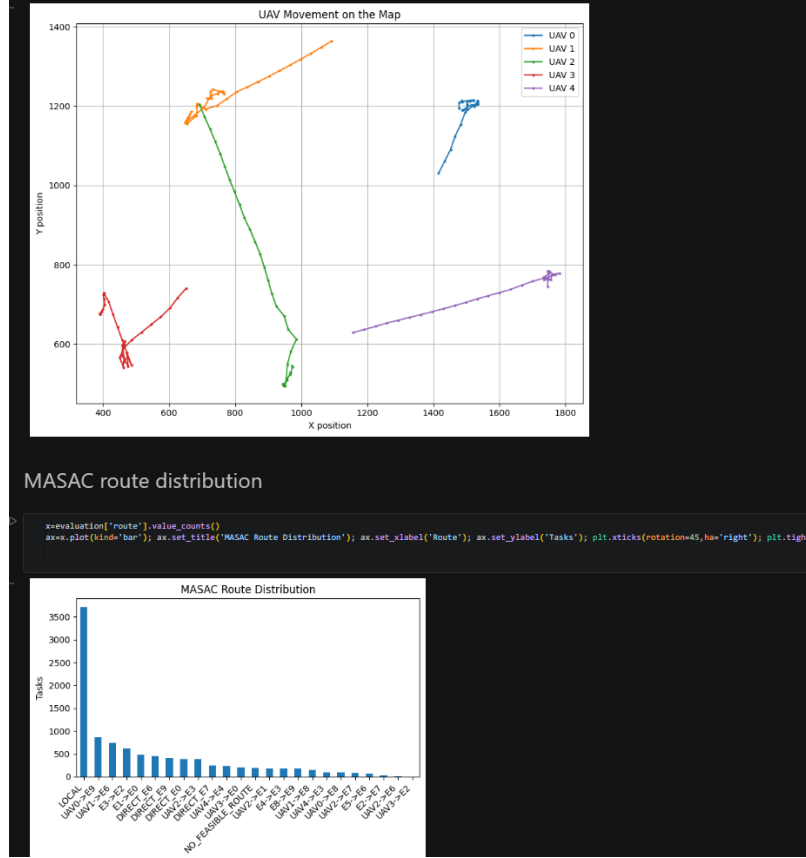


Figure 32: Set of UAV Movement and decision-distribution plots

15 Interpretation of Results

After inserting the final plots, interpretation should be based on relationships among metrics rather than only on whether each bar is small or large. For example:

- A reduction in Latency on UAV Routes is desirable only if it does not create a disproportionate increase in Energy or Packet Loss.
- Edge Load Distribution indicates how effectively the Policy prevents Workload from concentrating on a single Edge.
- Decision Distribution should be consistent with environment conditions; LOCAL in this plot is a processing decision and should not be presented as a fourth communication route.
- UAV Trajectory should show changes in UAV positions together with Traffic Hotspots; a constant count of 5 UAVs alone is not a useful indicator of Positioning quality.
- The Matched UAV/No-UAV Plot compares the two Route families on identical Tasks and is therefore fairer for analyzing the effect of Relaying than a comparison over two different Task sets.

16 Limitations and Scope of Implementation Claims

For an accurate report, several points must remain explicit:

- Cloud appears in the general architectural description, but the current Action Space does not implement an independent CLOUD Action or a Cloud processing Queue.
- In the current version, UAVs are communication Relays; a Compute Server on the UAV is not modeled.
- The MASAC implementation does not construct a simultaneous Joint Action for all Vehicles; decisions are made sequentially at the Task level, and the Actor is Shared across Agents.
- Packet Loss is a numerical SINR-based Surrogate because the input model does not provide an independent Packet-Loss formula.
- Numerical values such as CPU Frequencies, Energy coefficients, the 300- and 500-meter radii, and Reward Weights are experimental parameters and are not presented as values imposed by the source mathematical relations.

17 Conclusion

The project provides a complete Pipeline from temporal Vehicle/Task data to Policy learning and evaluation. The environment assigns measurable Consequences to each decision through real queues and UAV Mobility. The Action Mask applies physical constraints before Policy selection. The Actor constructs a discrete distribution over valid Actions and samples from it during Training. Using the Global State and Bellman Target, two Critics learn the Long-Term Value of decisions. The Actor then uses the Critics' Q-values together with Entropy Regularization to increase the Probability of better Actions. Target Critics stabilize the learning target through Soft Updates.

During Deployment, Critics are not required for direct Action selection; the trained Actor produces decisions using the Observation and Action Mask. Evaluation additionally creates a Matched comparison between UAV and No-UAV cases and supplies the final Notebook with Metrics for analyzing Latency, Energy, Packet Loss, Rate, Load Balancing, Success, and UAV Mobility.

Sources and Basis of the Report

1. The active VEC-IoT MASAC project code, including the principal files:
`src/masac.py`, `src/masac_env.py`, `src/uav_control.py`, `src/system_model.py`,
`src/xml_loader.py`, `train_masac.py`, and `evaluate_masac.py`.
2. The system model and communication/computation relations supplied with the project.
3. The final Notebook `VEC_IoT_MASAC_Complete_Colab_Matched_Final.ipynb` for Training, Evaluation, and result plotting.